Tutorial: Cómo usar el autocorrector

Estructura de Datos y Algoritmia — Curso 2026-2027

Este tutorial enseña paso a paso a:

- Ejecutar las **pruebas automáticas** de una práctica
- Interpretar la **salida** de las pruebas
- **Depurar** el código cuando una prueba falla
- Usar el autocorrector con prácticas en **C++ y en Java**

Al terminar sabrás utilizar el autocorrector para cualquier práctica de la asignatura **y para los exámenes prácticos**, que usan exactamente el mismo sistema.

> **Importante:** las prácticas se entregan antes del examen para disponer en él de tu propia implementación. La práctica no se evalúa directamente mediante la entrega, su evaluación se realiza mediante el **examen práctico**, donde recibirás tu código y una ampliación que deberás implementar. —

Estructura del proyecto

Cada práctica tiene la siguiente estructura:

```
practicaN/
  Makefile            ← compila el programa
  run_tests.sh        ← ejecuta las pruebas automáticas
  src/                ← tu código (aquí trabajas)
  data/               ← ficheros de datos de ejemplo
  tests/              ← pruebas automáticas (NO modificar las que se   ↪
  proporcionan)
    output/           ← salida de cada prueba (se genera automáticamente)
      *.valgrind.txt  ← log de la comprobación de memoria de cada prueba
```

Regla importante: todo tu código debe estar dentro de `src/`.

> **No modifiques las pruebas de los grupos A, B, C...** Modificar estas pruebas puede impedir que detectes errores importantes en tu código. Más adelante en este tutorial aprenderás a escribir tus propias pruebas.

En este tutorial, `src/` contiene una clase `Counter` con un par de errores deliberados y un método sin implementar. Tu objetivo es encontrarlos y corregirlos siguiendo la información proporcionada por las pruebas.

Las pruebas automáticas

Las pruebas de este ejemplo están organizadas en **dos grupos**:

Grupo	Qué prueba
A	La clase `Counter`
B	El programa `main.cc`

Cada práctica podrá tener una cantidad diferente de grupos y de número de pruebas en cada grupo, dependiendo de su complejidad y el número de ficheros de código fuente a probar.

Además, existe un **grupo Z** reservado para pruebas que crees tú mismo (ver Paso 9). Este grupo no se ejecuta por defecto.

Cada prueba puede tener **cinco resultados**:

Resultado	Significado
PASS (verde)	La prueba compila y pasa correctamente
FAIL (rojo)	La prueba compila pero alguna comprobación falla
ERROR (morado)	La prueba no compila (error de compilación)
CRASH (cian)	El programa terminó abruptamente (p. ej. *segmentation fault*, doble liberación de memoria)
TIMEOUT (amarillo)	La prueba superó el límite de tiempo (bucle infinito o algoritmo demasiado lento)

Los resultados **CRASH** y **TIMEOUT** son especialmente importantes en las prácticas con memoria dinámica y con algoritmos: indican errores de **ejecución** o de **eficiencia**, no de lógica.

Además, cada prueba en C++ se ejecuta también bajo **valgrind**, una herramienta que analiza el programa mientras corre y detecta errores de memoria que no siempre provocan un *crash* inmediato: lecturas o escrituras fuera de rango, uso de variables no inicializadas, fugas de memoria... Cuando valgrind encuentra un problema en una prueba, su fila del informe cambia a **MEMORY** en lugar de mostrar el resultado normal (más detalles en el Paso 10). Si valgrind no está instalado en tu equipo, `run_tests.sh` te avisa y continúa sin esta comprobación.

Paso 1 — Compilar el proyecto

Abre un terminal en la carpeta del proyecto y ejecuta:

```
make
```

Si todo compila correctamente, se genera el ejecutable `counter`. Si hay errores de compilación, verás los mensajes del compilador.

Paso 2 — Ejecutar todas las pruebas

Ejecuta el script de pruebas:

```
./run_tests.sh
```

Verás una barra de progreso seguida de un informe:

```
==========================================
  INFORME DE PRUEBAS
==========================================

  --- Grupo A: Counter ---
  PASS     test_A01_constructor_defecto
  PASS     test_A02_constructor_parametros
  PASS     test_A03_incrementar
  PASS     test_A04_decrementar
  FAIL     test_A05_decrementar_en_cero
  FAIL     test_A06_salida
  ERROR    test_A07_reset

  --- Grupo B: Programa ---
  FAIL     test_B01_ejecutar_programa

==========================================

  Aprobadas: 4  |  Fallidas: 3  |  Errores compilación: 1  |  Crash:  ↪
  0  |  Timeout: 0  |  Total: 8
  Errores de memoria (valgrind): 0

  Salida de cada prueba en: tests/output/
```

Hay **4 pruebas que fallan** (3 FAIL + 1 ERROR). El objetivo del tutorial es localizar los problemas y corregirlos.

Paso 3 — Ejecutar pruebas por grupo o individuales

No es necesario ejecutar todas las pruebas cada vez. Puedes lanzar un grupo o una prueba concreta:

```
./run_tests.sh A        # solo el grupo A (Counter)
./run_tests.sh B        # solo el grupo B (Programa)
./run_tests.sh A05      # solo la prueba A05
```

Paso 4 — Consultar la salida detallada de una prueba

La salida de cada prueba se guarda en `tests/output/`. Observa el contenido de los ficheros de esa carpeta para ver las salidas de las distintas pruebas.

Una prueba que **pasa** (PASS) muestra:

```
===============================================================
============
[doctest] test cases: 1 | 1 passed | 0 failed | 0 skipped
[doctest] assertions: 1 | 1 passed | 0 failed |
[doctest] Status: SUCCESS!
```

Una prueba que **falla** (FAIL) muestra qué comprobaciones fallaron:

```
===============================================================
============
tests/test_A05_decrementar_en_cero.cc:5:
TEST CASE:  A05: Decrementar en cero

tests/test_A05_decrementar_en_cero.cc:16: ERROR: CHECK( sePudo ==
(antes > 0) ) is NOT correct!
  values: CHECK( true == false )
  logged: decremento número 3
          antes := 0

tests/test_A05_decrementar_en_cero.cc:16: ERROR: CHECK( sePudo ==
(antes > 0) ) is NOT correct!
  values: CHECK( true == false )
  logged: decremento número 4
          antes := -1

tests/test_A05_decrementar_en_cero.cc:19: ERROR: CHECK( c.getValue()
== 0 ) is NOT correct!
  values: CHECK( -2 == 0 )

===============================================================
============
[doctest] test cases: 1 | 0 passed | 1 failed | 0 skipped
[doctest] assertions: 5 | 2 passed | 3 failed |
[doctest] Status: FAILURE!
```

De cada fallo se indica la **línea** del test, la **comprobación** que falló y los **valores** obtenidos (`values:`). Algunas pruebas añaden además una sección `logged:` con información sobre la situación concreta en la que se produjo el fallo; en el Paso 6 verás de dónde sale.

Y una prueba con **error de compilación** (ERROR) muestra el mensaje del compilador:

```
tests/test_A07_reset.cc: In function 'void DOCTEST_ANON_FUNC_14()':
```

```
tests/test_A07_reset.cc:7:7: error: 'class Counter' has no member      ↪
named 'reset'
    7 |     c.reset();
      |       ^~~~
```

> **Ojo:** el texto exacto de los mensajes depende del compilador y de su versión. El ejemplo anterior es de `g++` (el compilador que usa el autocorrector); si trabajas con `clang++`, en WSL, en un contenedor o en macOS, la redacción puede ser distinta (por ejemplo `error: no member named 'reset' in 'Counter'`). Lo importante es siempre lo mismo: **fichero**, **línea** y **causa** del error.

Una prueba con resultado **CRASH** o **TIMEOUT** incluye al final del fichero una explicación de las causas más habituales (punteros inválidos, dobles liberaciones, bucles infinitos...).

Paso 5 — Corregir el error de compilación (ERROR)

Empecemos por la prueba A07 que no compila. Abre el fichero del test `tests/test_A07_reset.cc`:

```
TEST_CASE("A07: Reset") {
    Counter c(42);
    c.reset();
    CHECK(c.getValue() == 0);
}
```

La prueba espera un método `reset()` que ponga el valor a cero. Pero ese método no existe todavía.

1. Abre `src/Counter.h` y añade la declaración del método `reset`:

   ```
   void reset();
   ```

2. Abre `src/Counter.cc` y añade la implementación:

```
void Counter::reset() {
    value = 0;
}
```

3. Ejecuta la prueba:

```
./run_tests.sh A07
```

```
  --- Grupo A: Counter ---
  PASS     test_A07_reset
```

¡Primer problema resuelto!

Paso 6 — Corregir un error lógico (FAIL)

Ahora abordamos la prueba A05. Lee su salida para ver los errores y el código del test en `tests/test_A05_decrementar_en_cero.cc` para entender qué se espera:

```
TEST_CASE("A05: Decrementar en cero") {
    Counter c(2);

    // Decrementamos más veces de las que se puede: al llegar a 0 el contador
    // debe quedarse en 0 y decrement() debe devolver false.
    for (int i = 1; i <= 4; ++i) {
        INFO("decremento número ", i);
        int antes = c.getValue();
        CAPTURE(antes);

        bool sePudo = c.decrement();
        CHECK(sePudo == (antes > 0));
    }

    CHECK(c.getValue() == 0);
}
```

La prueba parte de un contador con valor `2` y lo decrementa **cuatro veces**, es decir, dos veces más de las posibles. En cada vuelta comprueba que `decrement()` solo devuelve `true` cuando quedaba algo que decrementar (`antes > 0`), y al final que el contador se ha quedado en `0`.

Saber *en qué* situación falló: `INFO` y `CAPTURE`

Aquí el mismo `CHECK` se ejecuta cuatro veces, así que saber que ha fallado no basta: hay que saber en qué vuelta del bucle y con qué valores. Para eso *doctest* ofrece dos macros:

Macro	Qué hace
`INFO(...)`	Añade un mensaje de contexto; admite varios argumentos que se concatenan (texto, números, variables…)
`CAPTURE(var)`	Atajo para mostrar una variable con su nombre: imprime `var := valor`

Ambas son **pasivas**: no imprimen nada mientras las comprobaciones pasan, y solo valen dentro del bloque donde aparecen (aquí, la vuelta actual del bucle). Cuando un `CHECK` de ese bloque falla, doctest añade a la salida la sección `logged:` que viste en el Paso 4:

```
tests/test_A05_decrementar_en_cero.cc:16: ERROR: CHECK( sePudo ==      ↪
(antes > 0) ) is NOT correct!
  values: CHECK( true == false )
  logged: decremento número 3
          antes := 0
```

Ahora el diagnóstico es inmediato: los dos primeros decrementos van bien y el problema aparece **en el tercero**, cuando el contador ya valía `0` (`antes := 0`) y aun así `decrement()` devolvió `true`. Sin `INFO` ni `CAPTURE` solo sabrías que *alguna* de las cuatro vueltas falló.

Cuando escribas tus propias pruebas (Paso 9), usa estas dos macros siempre que un `CHECK` esté dentro de un bucle o dependa de varias variables: no cuestan nada y te ahorran mucho tiempo de depuración.

Corregir el error

Abre `src/Counter.cc` y busca el método:

```
bool Counter::decrement() {
    // BUG: no comprueba si value es 0
    value--;
    return true;
}
```

El problema es claro: **siempre decrementa y devuelve `true`**, sin comprobar si el valor ya es cero. Corrígelo:

```
bool Counter::decrement() {
    if (value <= 0) return false;
    value--;
    return true;
}
```

Verifícalo:

```
./run_tests.sh A05
```

La prueba pasa y la sección `logged:` desaparece: el contexto de `INFO` y `CAPTURE` solo se imprime cuando algo falla, así que puedes dejarlo puesto en tus pruebas sin ensuciar la salida.

Paso 7 — Corregir el formato de salida (FAIL)

Queda la prueba A06. Consulta su salida en el fichero `tests/output/test_A06_salida.txt`.

```
tests/test_A06_salida.cc:10: ERROR: CHECK( oss.str() == "Counter: 5"   ↪
) is NOT correct!
  values: CHECK( Counter = 5 == Counter: 5 )
```

La prueba esperaba `"Counter: 5"` pero obtuvo `"Counter = 5"`. Busca el `operator<<` en `src/Counter.cc` y cambia `" = "` por `": "`:

```
std::ostream& operator<<(std::ostream& os, const Counter& c) {
    os << "Counter: " << c.value;
    return os;
}
```

Paso 8 — Verificar que todo está correcto

Ejecuta todas las pruebas:

```
./run_tests.sh
```

```
  Aprobadas: 8  |  Fallidas: 0  |  Errores compilación: 0  |  Crash:  ↪
  0  |  Timeout: 0  |  Total: 8
```

¡Todas las pruebas pasan! Fíjate en que corregir el `operator<<` (prueba A06) también ha arreglado la prueba B01, que ejecuta el programa y comprueba su salida.

Paso 9 — Crear tus propias pruebas

Las pruebas proporcionadas no cubren el 100% de los casos. Parte de tu trabajo como programador es identificar qué situaciones no están probadas y escribir pruebas adicionales para ganar confianza en tu código. Esta es la mejor forma de **prepararte para el examen**.

Tus pruebas van en el **grupo Z**. Este grupo **no se ejecuta por defecto**, solo se lanza con `./run_tests.sh Z`.

9.1 — Identificar huecos en las pruebas

Repasa las pruebas existentes y piensa qué escenarios faltan. Por ejemplo:

Funcionalidad	Pruebas existentes	¿Qué falta?
`Counter()`	A01: valor inicial es 0	—
`Counter(int)`	A02: valor positivo (5)	¿Qué pasa con valores negativos?
`increment()`	A03: incrementar desde 0	¿Funciona desde un valor distinto de 0?
`decrement()`	A04: desde 3; A05: hasta pasarse de 0	—

9.2 — Escribir una prueba nueva (C++)

Crea un fichero `tests/test_Z01_incrementar_desde_valor.cc` con el siguiente contenido:

```
#define DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN
#include "common/doctest.h"
#include "Counter.h"

TEST_CASE("Z01: Incrementar desde valor inicial") {
    Counter c(10);
    c.increment();
    CHECK(c.getValue() == 11);
}
```

La estructura es siempre la misma:

1. Las dos primeras líneas configuran el framework de pruebas (*doctest*).
2. Se incluye la cabecera de la clase que se quiere probar.
3. `TEST_CASE("nombre")` define el caso de prueba.
4. Dentro, se usa `CHECK(expresión)` para verificar condiciones. Si la expresión es falsa, la prueba falla y se muestra qué valor se obtuvo frente al esperado. Recuerda `INFO` y `CAPTURE` (Paso 6) si el `CHECK` está dentro de un bucle o depende de varias variables.

Importante: el nombre del fichero debe empezar por `test_Z` seguido de un número de dos cifras (Z01, Z02...). Solo así el autocorrector lo reconocerá como parte del grupo Z.

9.3 — Ejecutar las pruebas del grupo Z

```
./run_tests.sh Z        # todas tus pruebas
./run_tests.sh Z01      # una prueba concreta
```

9.4 — Ejercicio

Escribe otra prueba en `tests/test_Z02_constructor_negativo.cc` que compruebe qué ocurre al crear un `Counter` con un valor negativo. ¿Debería el contador aceptar valores negativos? ¿Qué devuelve `getValue()`? Decide cuál es el comportamiento correcto y escribe los `CHECK` correspondientes.

Paso 10 — Detectar y corregir errores de memoria con valgrind

Hasta ahora hemos corregido errores de compilación y de lógica. Pero hay una categoría de errores que ni el compilador ni un `CHECK` que pase por casualidad detectan: los **errores de memoria**. Un programa puede escribir fuera de los

límites de un array, leer una variable que nunca se inicializó, o dejar memoria sin liberar, y aun así parecer que "funciona" en una ejecución concreta.

Valgrind es una herramienta que ejecuta tu programa bajo un intérprete instrumentado y vigila cada acceso a memoria. Cuando detecta algo incorrecto, lo reporta aunque el programa no llegue a fallar (*crash*) y aunque las comprobaciones (`CHECK`) den el resultado esperado.

A continuación se proporcionan dos pruebas nuevas para el grupo Z, con errores de memoria reales para que las ejecutes y las corrijas. **No usan punteros** para que veas que estos errores también aparecen con `std::vector` o con variables locales normales.

> **Ojo:** los logs que se muestran a continuación se generaron con g++ 13 y valgrind 3.22 sobre Linux. En tu equipo los números de proceso (`==92==`), las direcciones de memoria (`0x...`) y hasta el recuento del `ERROR SUMMARY` serán distintos, y cambian de una ejecución a otra. Lo que no cambia es lo que hay que buscar: el **tipo de error** (`Invalid write`, `uninitialised value(s)`...) y la línea de **tu** fichero de test.

10.1 — Un acceso fuera de rango

Crea `tests/test_Z03_memoria_vector.cc` con este contenido:

```
#define DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN
#include "common/doctest.h"
#include <vector>

TEST_CASE("Z03: Historial de valores") {
    std::vector<int> historial(3);
    historial[0] = 10;
    historial[1] = 20;
    historial[2] = 30;
    // BUG: historial solo tiene posiciones 0, 1 y 2. Escribir en la
    // posición 3 escribe fuera del bloque reservado en el heap.
    historial[3] = 40;

    CHECK(historial[2] == 30);
}
```

El `historial` reserva sitio para 3 enteros (posiciones 0, 1 y 2), pero el código escribe también en la posición 3, fuera de ese bloque. Ejecútalo:

```
./run_tests.sh Z03
```

```
  --- Grupo Z: Pruebas del alumno ---
  MEMORY   test_Z03_memoria_vector                    test: PASS
```

Fíjate: el `CHECK` es correcto (`test: PASS`), pero la fila se marca como **MEMORY**. Un `PASS` no es suficiente si el programa toca memoria que no le

pertenece. Abre el log detallado, que está en `tests/output/` con el nombre `test_Z03_memoria_vector.valgrind.txt.NNN` (`NNN` es el número de proceso, y cambia en cada ejecución):

```
==92== Invalid write of size 4
==92==    at 0x1231E4: DOCTEST_ANON_FUNC_14()                              ↪
(test_Z03_memoria_vector.cc:12)
==92==    by 0x12270F: doctest::Context::run() (doctest.h:7007)
==92==    by 0x1230AF: main (doctest.h:7085)
==92==  Address 0x4e2596c is 0 bytes after a block of size 12 alloc'd
==92==    at 0x48859FC: operator new(unsigned long) (...)
                        [varias líneas de la implementación de           ↪
                        std::vector]
==92==    by 0x12317B: DOCTEST_ANON_FUNC_14()                              ↪
(test_Z03_memoria_vector.cc:6)
==92==    by 0x12270F: doctest::Context::run() (doctest.h:7007)
==92==    by 0x1230AF: main (doctest.h:7085)
==92== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

La línea clave es `Invalid write of size 4 ... (test_Z03_memoria_vector.cc:12)`: valgrind señala exactamente la línea del fichero donde se escribió fuera de rango. Corrige el bug (por ejemplo, reserva 4 posiciones o usa `push_back(40)` en lugar de un índice fijo) y vuelve a ejecutar `./run_tests.sh Z03`: la fila `MEMORY` debe desaparecer.

10.2 — Una variable no inicializada

Crea `tests/test_Z04_variable_no_inicializada.cc` con este contenido:

```cpp
#define DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN
#include "common/doctest.h"

TEST_CASE("Z04: Comprobar signo") {
    int numero = -5;
    bool es_positivo;  // BUG: no se inicializa

    if (numero > 0) {
        es_positivo = true;
    }
    // Como numero es negativo, la rama anterior no se ejecuta:
    // es_positivo se lee aquí sin haber sido asignada nunca.
    CHECK(es_positivo == false);
}
```

Como `numero` es negativo, la rama del `if` nunca se ejecuta, así que `es_positivo` se lee en el `CHECK` sin haber sido asignada nunca. Ejecuta:

```
./run_tests.sh Z04
```

```
  --- Grupo Z: Pruebas del alumno ---
  MEMORY   test_Z04_variable_no_inicializada        test: FAIL
```

El resultado del `CHECK` es pura casualidad: depende de qué basura hubiera en la pila en ese momento. Aquí ha salido `FAIL` (la basura resultó ser `true`), pero en otra máquina, con otro compilador o incluso en otra ejecución podría salir `PASS` y hacerte creer que el código es correcto. **Esto es precisamente lo peligroso de una variable no inicializada**, y por eso hace falta una herramienta como valgrind que lo detecte siempre, independientemente de lo que muestre el `CHECK`. Abre el log `tests/output/test_Z04_variable_no_inicializada.valgrind.txt.NNN`:

```
==119== Conditional jump or move depends on uninitialised value(s)
==119==    at 0x12CB78: ...                                                ↪
doctest::detail::Expression_lhs<bool&>::operator==<bool>(bool&&)           ↪
(doctest.h:1527)
==119==    by 0x1231F7: DOCTEST_ANON_FUNC_14()                             ↪
(test_Z04_variable_no_inicializada.cc:13)
==119==    by 0x12270F: doctest::Context::run() (doctest.h:7007)
==119==    by 0x1230AF: main (doctest.h:7085)
                        [8 bloques más, todos dentro de doctest.h]
==119== ERROR SUMMARY: 9 errors from 9 contexts (suppressed: 0 from 0)
```

Este log es más largo y menciona varias veces `doctest.h`, porque el valor sin inicializar viaja a través de la maquinaria interna de *doctest* antes de usarse en la comparación. No te distraigas con esas líneas: busca la que sí menciona **tu** fichero de test —aquí, `test_Z04_variable_no_inicializada.cc:13`, la línea del `CHECK`— esa es la pista útil. Corrige el bug dando un valor inicial a la variable (`bool es_positivo = false;`) y vuelve a ejecutar `./run_tests.sh Z04`: la fila `MEMORY` debe desaparecer.

Paso 11 — El autocorrector con Java

La práctica 3 de este curso se realiza en **Java**, y el autocorrector funciona exactamente igual: mismos comandos, mismos grupos, mismos resultados. La única diferencia es cómo se escriben las pruebas.

En la carpeta `ejemplo-java/` de este tutorial tienes un mini-proyecto Java con la misma clase `Counter` (y el mismo error deliberado en `decrement()`). Pruébalo:

```
cd ejemplo-java
./run_tests.sh
```

Verás que la prueba `test_A04_decrementar_en_cero` falla. Abre `src/Counter.java`, corrige el método `decrement()` igual que hiciste en C++ y vuelve a ejecutar las pruebas.

Pruebas en Java

Las pruebas Java son ficheros `tests/test_XNN_nombre.java`. En lugar de *doctest* se usa la clase auxiliar `Check` (en `tests/common/Check.java`):

```java
public class test_Z01_incrementar_desde_valor {
    public static void main(String[] args) {
        Counter c = new Counter(10);
        c.increment();
        Check.equals(11, c.getValue(), "Z01: incrementar desde 10");
        Check.end();   // imprime el resumen - no lo olvides
    }
}
```

Reglas:

1. El nombre de la **clase** debe coincidir con el nombre del **fichero**.
2. Las comprobaciones disponibles son `Check.equals`, `Check.isTrue`, `Check.isFalse` y `Check.contains`.
3. La última línea debe ser siempre `Check.end()`.

La salida detallada se guarda igualmente en `tests/output/`.

Resumen de comandos

Comando	Qué hace
`make`	Compila el programa desde `src/`
`make clean`	Elimina los binarios compilados
`./run_tests.sh`	Ejecuta **todas** las pruebas
`./run_tests.sh A`	Ejecuta las pruebas del grupo A
`./run_tests.sh A05`	Ejecuta solo la prueba A05
`./run_tests.sh Z`	Ejecuta las pruebas del alumno (grupo Z)

Flujo de trabajo recomendado

Cuando trabajes en una práctica, sigue estos pasos:

1. **Implementa** una parte del código en `src/`.
2. **Ejecuta** las pruebas del grupo correspondiente (`./run_tests.sh A`).
3. Si hay fallos, **consulta la salida** detallada de las pruebas.
4. **Lee el código del test** para entender qué se espera.
5. **Corrige** el error y vuelve a ejecutar la prueba individual (`./run_tests.sh A05`).
6. Cuando el grupo pasa, **avanza** al siguiente grupo.
7. Al final, ejecuta **todas** las pruebas (`./run_tests.sh`).
8. **Escribe pruebas del grupo Z** para los casos que las pruebas básicas no cubren: es tu mejor preparación para el examen.

Prepararse para el examen: en el examen dispondrás de tu práctica, las pruebas completas y una ampliación, sin Internet ni asistentes de ningún tipo. La única preparación eficaz es haber **escrito y comprendido** tu propio código. Si hoy resuelves la práctica copiando código que no entiendes, el examen lo pondrá en evidencia.